

Experiment No.	5
Experiment Title	Decision Tree and Random Forest: A Comparative Classification Study
Student Name	Hemnath D
Register Number	3122247001022
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	16 August 2026
GitHub Repository	https://github.com/hemnathd12/ml-assignments

1 Objective

To build and compare a Decision Tree classifier and a Random Forest ensemble on the Wisconsin Diagnostic Breast Cancer dataset, tune their hyperparameters using 5-fold cross-validation, and evaluate which model generalises better to unseen data.

2 Problem Statement

Breast cancer diagnosis from digitised fine needle aspirate (FNA) images can be framed as a binary classification problem: given 30 numerical features describing the size, shape and texture of cell nuclei, predict whether a tumour is malignant or benign. The input to the system is the 30-dimensional feature vector for a sample, and the output is a class label (Malignant/Benign) along with a probability score. The objective of this experiment is to train a single Decision Tree and a Random Forest ensemble on this data, tune both models using cross-validation, and quantitatively compare their classification performance and their tendency to overfit.

3 Dataset Description

Dataset Name	Wisconsin Diagnostic Breast Cancer (WDBC)
Dataset Source	UCI Machine Learning Repository (https://archive.ics.uci.edu), also bundled with scikit-learn as <code>load_breast_cancer</code>
Number of Samples	569
Number of Features	30 numerical, real-valued features (mean, standard error, and “worst” value of 10 base measurements per cell nucleus)
Number of Classes	2 (Malignant = 212 samples, Benign = 357 samples)
Missing Values	None (0 missing entries across all 30 features)
Train-Test Split	80% train (455 samples) / 20% test (114 samples), stratified on class label

4 Exploratory Data Analysis

4.1 Class Distribution

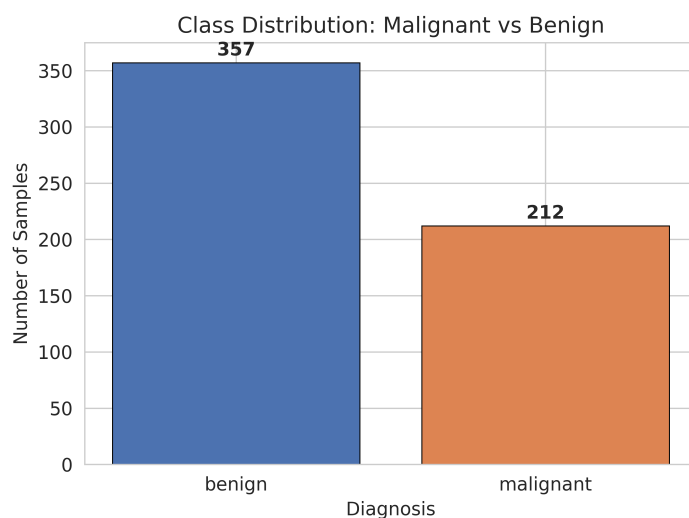


Figure 1: Class distribution of malignant vs. benign samples in the WDBC dataset

Inference:

The dataset is moderately imbalanced, with benign cases (357) outnumbering malignant cases (212) by roughly a 1.7:1 ratio. This imbalance is not severe enough to require resampling techniques such as SMOTE, but it does justify using stratified splitting and stratified k-fold cross-validation so that both classes are proportionally represented in every fold. It also motivates looking beyond raw accuracy to metrics such as precision, recall, and F1-score, since a model that is biased toward the majority (benign) class could still report a deceptively high accuracy.

4.2 Correlation Matrix

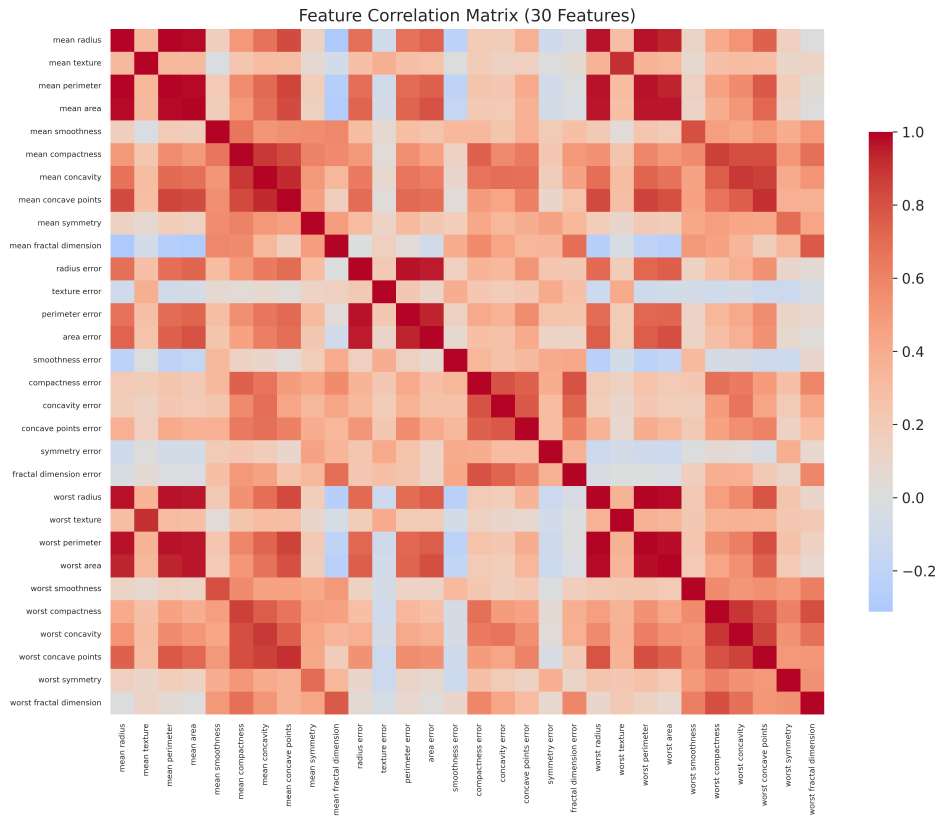


Figure 2: Pairwise Pearson correlation matrix across all 30 features

Inference:

Several clusters of strongly correlated features are visible, particularly among the “mean”, “standard error” and “worst” variants of the same base measurement (e.g., radius, perimeter, and area are almost perfectly correlated with one another since they are geometrically related). This multicollinearity is not a major concern for tree-based models such as Decision Trees and Random Forests, because they select one feature to split on at a time rather than fitting a linear combination of all features. However, it does explain why a small subset of features tends to dominate the feature-importance rankings later in the experiment, since correlated features carry largely redundant information.

4.3 Feature Distributions

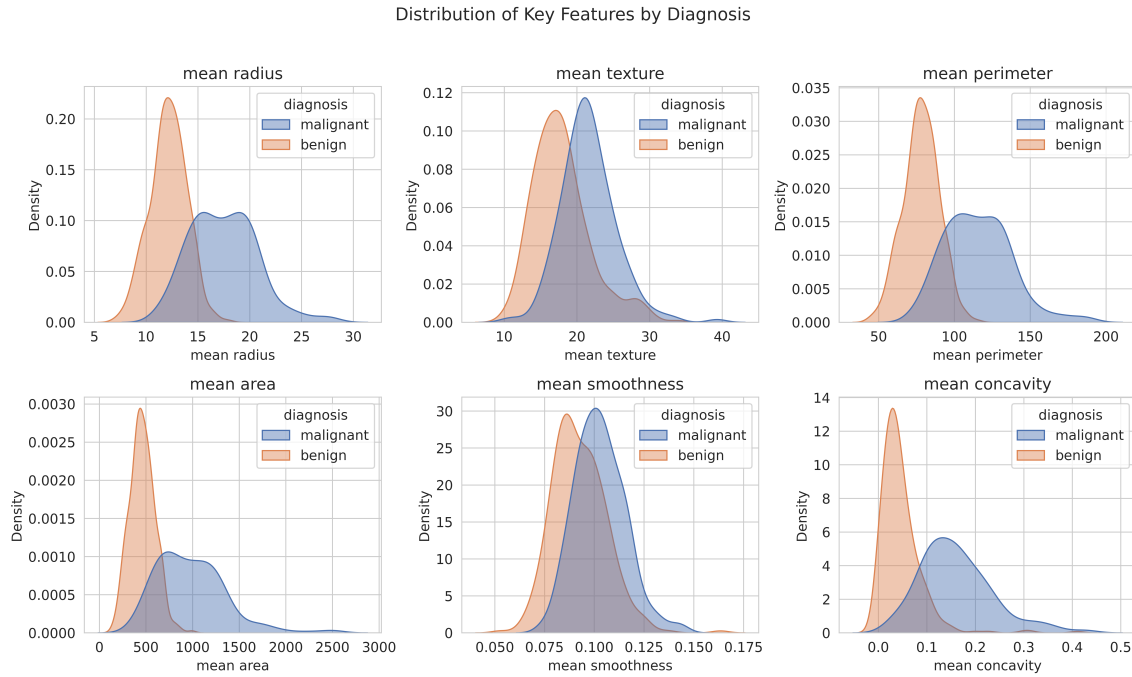


Figure 3: Kernel density estimates of six representative mean-value features, split by diagnosis

Inference:

For nearly all of the plotted features, the distribution for malignant samples is shifted to the right of the benign distribution, indicating that malignant tumours tend to have larger radius, perimeter, area, and concavity values on average. Mean smoothness shows the greatest overlap between the two classes, suggesting it is individually a weaker discriminator, whereas mean concavity and mean area show cleaner separation. This pattern is consistent with clinical intuition: malignant cell nuclei are typically larger and more irregularly shaped than benign ones.

4.4 Outlier Analysis

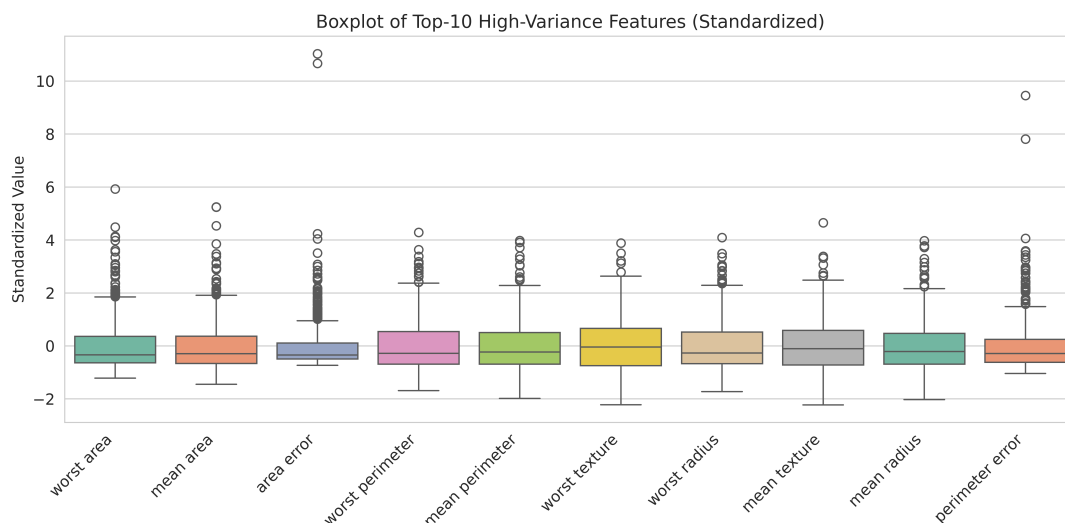


Figure 4: Boxplots of the ten highest-variance features after standardisation

Inference:

The standardised boxplots reveal a noticeable number of points beyond the whiskers for several features, particularly the “worst” and “standard error” variants of area and perimeter, indicating the presence of outlier samples with unusually extreme measurements. Rather than removing these points, they were retained in the analysis because tree-based ensembles are relatively robust to outliers – splits are based on ranking or thresholding a feature rather than on its exact magnitude, so a small number of extreme values does not distort the decision boundary the way it would for a distance-based or linear model.

5 Data Preprocessing

The following preprocessing steps were applied before model training:

- **Missing value handling:** The dataset was checked for missing entries; none were found (an ID column present in the original UCI release was already dropped in the scikit-learn version used), so no imputation was necessary.
- **Encoding:** The target labels (**malignant/benign**) were converted to a binary numeric label using `LabelEncoder`, with benign mapped to 0 and malignant mapped to 1.
- **Feature scaling:** All 30 features were standardised using `StandardScaler` (zero mean, unit variance), fit only on the training set and then applied to the test set to avoid data leakage. While tree-based models are scale-invariant in principle, standardisation was applied for consistency across the pipeline and to keep the preprocessing reusable for any distance-based baseline comparisons.
- **Train-test split:** An 80–20 stratified split was used so that the 1.7:1 class ratio was preserved in both the training and test partitions.

- **Feature engineering:** No new features were engineered; all 30 original attributes were retained, since the correlation analysis showed that the redundancy among related features is naturally handled by the tree-splitting process rather than needing manual removal.

Code Snippet 1: Label encoding, scaling, and stratified train-test split.

```

1  le = LabelEncoder()
2  y = le.fit_transform(y_raw)    # benign -> 0, malignant -> 1
3
4  X_train, X_test, y_train, y_test = train_test_split(
5      X, y, test_size=0.20, random_state=42, stratify=y
6  )
7
8  scaler = StandardScaler()
9  X_train_s = scaler.fit_transform(X_train)    # fit only on training data
10 X_test_s = scaler.transform(X_test)         # avoid data leakage

```

6 Machine Learning Algorithm

6.1 Decision Tree Classifier

A Decision Tree recursively partitions the feature space into axis-aligned regions by choosing, at each node, the feature and threshold that most reduce class impurity in the resulting child nodes.

Working principle: Starting from the root node containing all training samples, the algorithm evaluates every candidate feature/threshold pair and picks the split that maximises impurity reduction. This process repeats recursively on each child node until a stopping criterion (maximum depth, minimum samples per leaf, or pure leaf nodes) is reached.

Mathematical formulation: For a node t with class proportions $p_1, p_2, \dots, p_k$, the Gini impurity is

$$Gini(t) = 1 - \sum_{i=1}^k p_i^2,$$

and the entropy is

$$H(t) = - \sum_{i=1}^k p_i \log_2 p_i.$$

A candidate split partitions node t into children t_L and t_R ; the split is chosen to maximise the information gain

$$\Delta = I(t) - \frac{N_L}{N} I(t_L) - \frac{N_R}{N} I(t_R),$$

where $I(\cdot)$ is either the Gini impurity or entropy, and N_L, N_R are the number of samples routed to each child.

Advantages: Highly interpretable (the learned rules can be visualised and read directly), requires little preprocessing, handles non-linear decision boundaries, and naturally supports both numerical and categorical features.

Limitations: A single deep tree tends to overfit the training data, memorising noise as if it were signal, which leads to high variance – small changes in the training set can produce very different trees. Shallow trees, on the other hand, can underfit and miss important interactions between features.

6.2 Random Forest Classifier

A Random Forest is an ensemble of many decision trees, each trained on a different bootstrap sample of the training data and considering only a random subset of features at each split.

Working principle: Bootstrap aggregation (bagging) creates B resampled training sets by sampling with replacement; a separate decision tree is grown on each. To further decorrelate the trees, only a random subset of $m \approx \sqrt{p}$ features (for classification, where p is the total number of features) is considered at every split. Predictions from all B trees are combined by majority vote.

Mathematical formulation: For an input x , each tree T_b produces a class prediction $\hat{y}_b(x)$. The forest prediction is

$$\hat{y}(x) = \text{mode}\{\hat{y}_1(x), \hat{y}_2(x), \dots, \hat{y}_B(x)\},$$

and the predicted class probability used for the ROC/PR curves in this experiment is the fraction of trees voting for the positive class,

$$\hat{p}(x) = \frac{1}{B} \sum_{b=1}^B \mathbb{I}[\hat{y}_b(x) = 1].$$

Advantages: Averaging over many decorrelated trees substantially reduces variance compared to a single tree, improves generalisation, and provides a natural feature-importance ranking as a by-product of training.

Limitations: Loses the direct interpretability of a single tree (though feature importance partially compensates), is more computationally expensive to train and to run inference with, and can still overfit if the individual trees are allowed to grow arbitrarily deep with too little regularisation.

7 Experimental Setup

Python Version	3.12
Scikit-Learn Version	1.5
IDE	Jupyter / VS Code
Operating System	Ubuntu 24.04 LTS
Hardware	Standard CPU-only compute environment

8 Hyperparameter Selection

Both models were tuned using an exhaustive grid search with **5-fold stratified cross-validation** on the training set, optimising for F1-score (chosen over raw accuracy because

it balances precision and recall, which matters for a medical diagnosis task where false negatives are costly).

Decision Tree search space: criterion (gini or entropy), max_depth (3, 4, 5, 6, 8, 10, or unrestricted), min_samples_split (2, 5, or 10), and min_samples_leaf (1, 2, or 4) – a total of 126 combinations.

Random Forest search space: n_estimators (100, 200, or 300), max_depth (5, 8, 10, or unrestricted), max_features (sqrt or log2), and bootstrap (True or False) – a total of 48 combinations.

Code Snippet 2: Grid search with 5-fold stratified cross-validation.

```

1  cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
2
3  dt_param_grid = {
4      "criterion": ["gini", "entropy"],
5      "max_depth": [3, 4, 5, 6, 8, 10, None],
6      "min_samples_split": [2, 5, 10],
7      "min_samples_leaf": [1, 2, 4],
8  }
9  dt_grid = GridSearchCV(DecisionTreeClassifier(random_state=42),
10                        dt_param_grid, cv=cv, scoring="f1", n_jobs=-1)
11  dt_grid.fit(X_train_s, y_train)
12  best_dt = dt_grid.best_estimator_
13
14  rf_param_grid = {
15      "n_estimators": [100, 200, 300],
16      "max_depth": [5, 8, 10, None],
17      "max_features": ["sqrt", "log2"],
18      "bootstrap": [True, False],
19  }
20  rf_grid = GridSearchCV(RandomForestClassifier(random_state=42,
21      ↪ n_jobs=-1),
22                        rf_param_grid, cv=cv, scoring="f1", n_jobs=-1)
23  rf_grid.fit(X_train_s, y_train)
24  best_rf = rf_grid.best_estimator_

```

The best combination found for each model, based on mean cross-validated F1-score, is summarised below.

Parameter	Value
DT – criterion	entropy
DT – max_depth	4
DT – min_samples_split	2
DT – min_samples_leaf	2
RF – n_estimators	200
RF – max_depth	10
RF – max_features	sqrt
RF – bootstrap	False

8.1 Decision Tree Cross-Fold Results

Table 2: Decision Tree Hyperparameter Evaluation using 5-Fold Cross-Validation (best-performing combination shown per criterion/depth pair, sorted by accuracy)

Criterion	Max Depth	Avg CV Accuracy (%)	Avg CV F1 Score
entropy	4	94.07	0.9163
entropy	5	93.63	0.9121
entropy	6	93.41	0.9094
entropy	None	93.41	0.9094
entropy	10	93.41	0.9094
entropy	8	93.41	0.9094
gini	6	93.19	0.9067
gini	10	92.97	0.9039
gini	8	92.97	0.9039
gini	None	92.97	0.9039
gini	3	92.75	0.8990
gini	4	92.53	0.8949
entropy	3	92.53	0.8953

Inference:

Accuracy rises as the tree is allowed to grow from depth 3 to depth 4 and then plateaus, or very slightly declines, beyond depth 5–6. This is the expected overfitting signature: a very shallow tree underfits by not capturing enough interactions, while an unconstrained tree (`max_depth = None`) offers no accuracy benefit over a depth of 4–6 on the validation folds, because the additional splits mostly fit noise rather than genuine signal. The entropy criterion consistently edges out Gini impurity in this search, though the difference is small (typically under 1 percentage point), indicating that the choice of impurity measure is a secondary factor compared to controlling tree depth.

8.2 Random Forest Cross-Fold Results

Table 3: Random Forest Hyperparameter Evaluation using 5-Fold Cross-Validation (top combinations by accuracy)

n_estimators	Max Depth	Max Features	Avg CV Acc. (%)	Avg CV F1
200	10	sqrt	97.14	0.9615
200	None	sqrt	97.14	0.9615
200	5	sqrt	97.14	0.9608
300	5	sqrt	97.14	0.9608
300	None	log2	96.92	0.9585
100	5	sqrt	96.92	0.9580
100	10	log2	96.92	0.9585
300	8	log2	96.92	0.9586
100	None	log2	96.70	0.9556
300	10	log2	96.70	0.9556

Inference:

Unlike the Decision Tree, Random Forest accuracy is remarkably stable across nearly every combination of depth and estimator count, with almost all configurations clustering between 96% and 97% cross-validated accuracy. This flatness illustrates the core benefit of ensembling: because each tree in the forest is trained on a bootstrap sample with a random feature subset, no single tree’s overfitting can dominate the majority vote, so the model is far less sensitive to hyperparameter choice than a standalone tree. The best configuration (200 trees, `max_depth = 10`, `max_features='sqrt'`, no bootstrap subsampling) was selected for the final model, though several nearby configurations perform almost identically, confirming that the ensemble has reached a stable performance plateau rather than a narrow optimum.

8.3 5-Fold Cross-Validation Performance Comparison

Table 4: 5-Fold Cross-Validation Accuracy Comparison (Training Set)

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Average
Decision Tree	93.41	95.60	90.11	95.60	95.60	94.07
Random Forest	96.70	97.80	95.60	96.70	98.90	97.14

Inference:

The Random Forest outperforms the Decision Tree on every single fold, not just on average, which rules out the possibility that its higher mean accuracy is driven by a lucky split. The Decision Tree’s fold-to-fold accuracy also swings more widely (from 90.11% to 95.60%, a range of 5.49 points) compared to the Random Forest’s tighter range (95.60% to 98.90%, 3.30 points). This lower variance across folds is direct empirical evidence of the variance-reduction property of bagging: averaging predictions across many decorrelated

trees smooths out the sensitivity that a single tree has to exactly which samples happen to fall into its training fold.

9 Results

Both tuned models were retrained on the full training set (455 samples) and evaluated once on the untouched 20% held-out test set (114 samples).

Code Snippet 3: Final evaluation on the held-out test set.

```

1 best_dt.fit(X_train_s, y_train)
2 best_rf.fit(X_train_s, y_train)
3
4 def evaluate(model, X_te, y_te):
5     y_pred = model.predict(X_te)
6     y_prob = model.predict_proba(X_te)[: , 1]
7     return {
8         "accuracy": accuracy_score(y_te, y_pred),
9         "precision": precision_score(y_te, y_pred),
10        "recall": recall_score(y_te, y_pred),
11        "f1": f1_score(y_te, y_pred),
12        "y_pred": y_pred, "y_prob": y_prob,
13    }
14
15 dt_eval = evaluate(best_dt, X_test_s, y_test)
16 rf_eval = evaluate(best_rf, X_test_s, y_test)

```

Metric	Decision Tree	Random Forest
Accuracy	92.98%	97.37%
Precision	100.00%	100.00%
Recall	80.95%	92.86%
F1-score	89.47%	96.30%
ROC-AUC	95.63%	99.40%

10 Result Visualization

10.1 Confusion Matrices

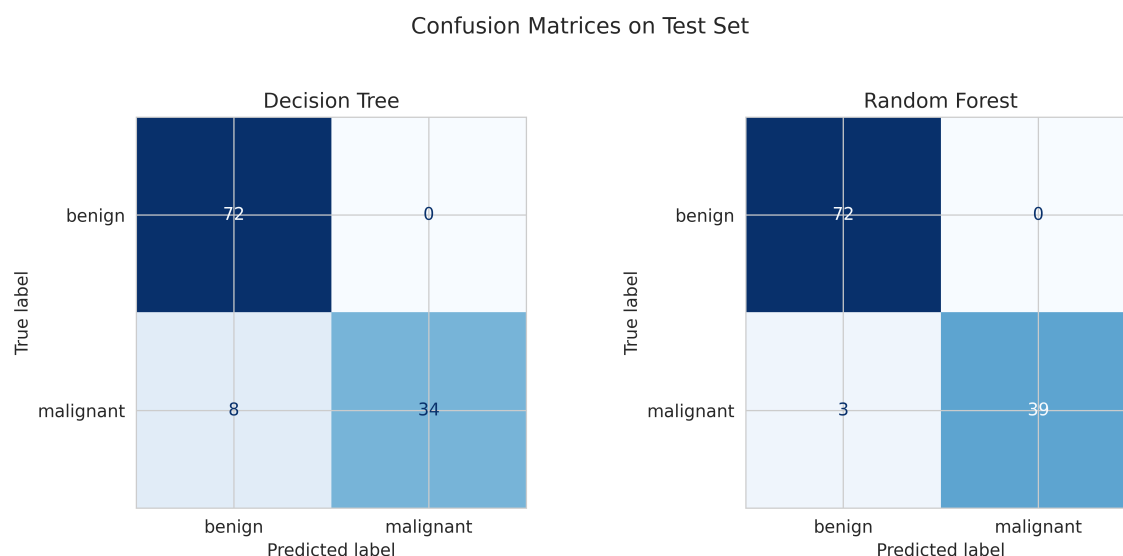


Figure 5: Confusion matrices for the Decision Tree (left) and Random Forest (right) on the test set

Inference:

Both models achieve perfect precision on this test split (zero false positives), meaning that every sample predicted as malignant was truly malignant. The key difference lies in recall: the Decision Tree misses 8 malignant cases (classifying them as benign), whereas the Random Forest misses only 3. In a medical screening context this is the most clinically important gap, since a false negative means a malignant tumour goes undetected, so the Random Forest's higher recall directly translates into fewer missed diagnoses.

10.2 ROC Curve

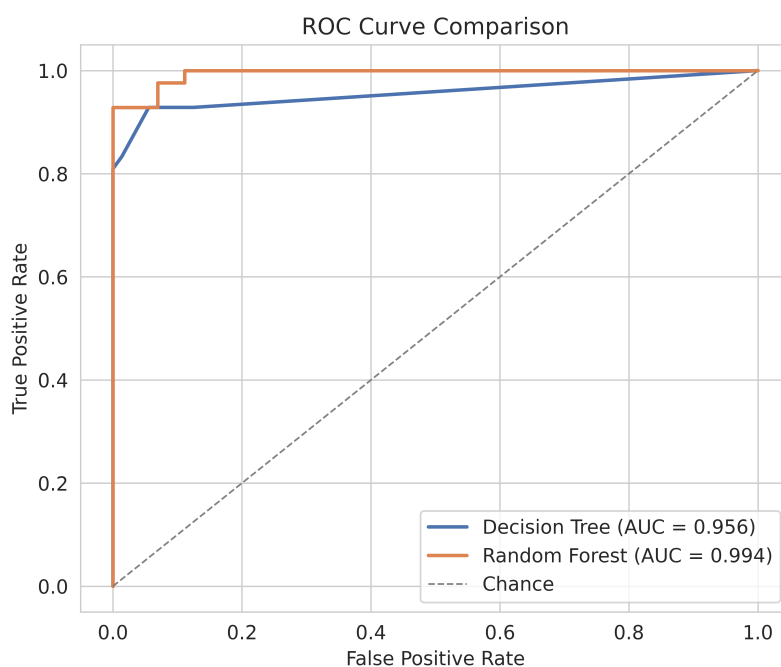


Figure 6: ROC curves for both models on the test set

Inference:

The Random Forest's ROC curve hugs the top-left corner far more tightly than the Decision Tree's, giving it a near-perfect AUC of 0.994 versus 0.956 for the single tree. The Decision Tree's curve is also visibly "blockier," a direct consequence of it having far fewer distinct probability thresholds (since a tree's leaves output a small number of discrete class proportions) compared to the smoother probability estimates the Random Forest produces by averaging over 200 trees.

10.3 Precision-Recall Curve

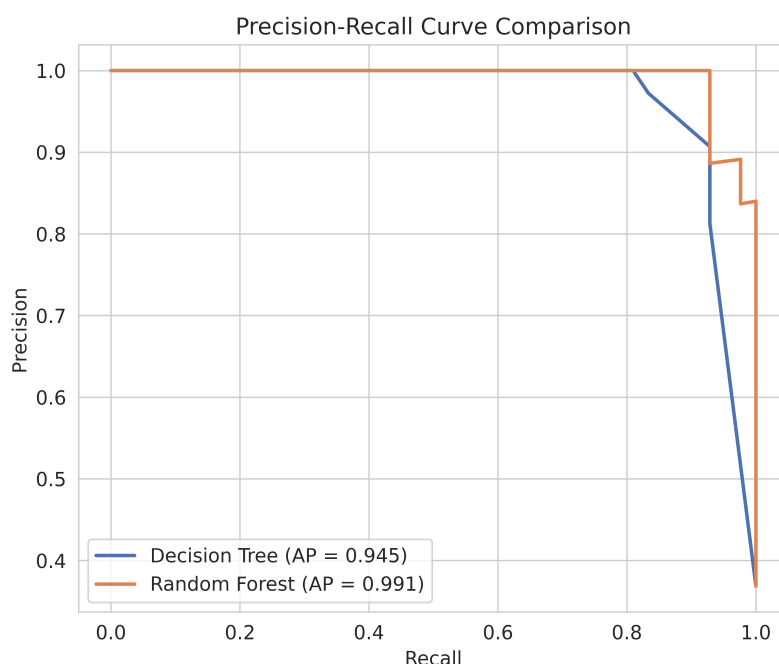


Figure 7: Precision-recall curves for both models on the test set

Inference:

The precision-recall curve is particularly informative here because the positive (malignant) class is the minority class. The Random Forest maintains precision close to 1.0 across almost the entire recall range, while the Decision Tree's precision starts to drop off earlier as recall increases. This confirms that the Random Forest's advantage is not just in overall accuracy but specifically in how well it separates the two classes even as the decision threshold is varied.

10.4 Learning Curves



Figure 8: Learning curves showing training and cross-validation accuracy as a function of training set size

Inference:

The Decision Tree's training score stays close to 1.0 across all training sizes while its validation score plateaus noticeably lower, and the persistent gap between the two curves is the textbook signature of overfitting/high variance. The Random Forest shows a much narrower gap between training and validation accuracy, and its validation curve is both higher and less jagged, confirming that the ensemble generalises better at every training set size tested rather than only at the final full-data point.

10.5 Feature Importance

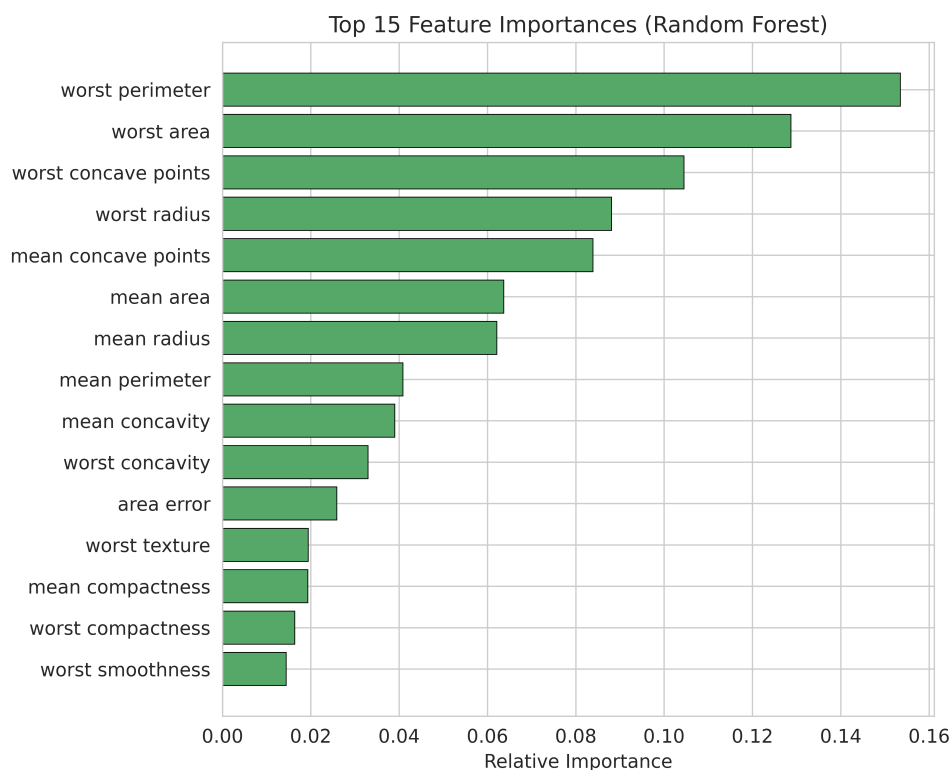


Figure 9: Top 15 feature importances from the tuned Random Forest

Inference:

“Worst concave points” and “worst perimeter” emerge as the two most influential features, followed closely by “worst radius” and “mean concave points.” This aligns with the EDA correlation analysis, which showed that the “worst” (largest observed) values of size- and shape-related measurements are strong discriminators between malignant and benign nuclei. Notably, the importances are spread across a handful of features rather than concentrated on a single one, which is a healthy sign that the Random Forest is drawing on multiple complementary measurements rather than relying on one potentially noisy feature.

10.6 Decision Tree Structure

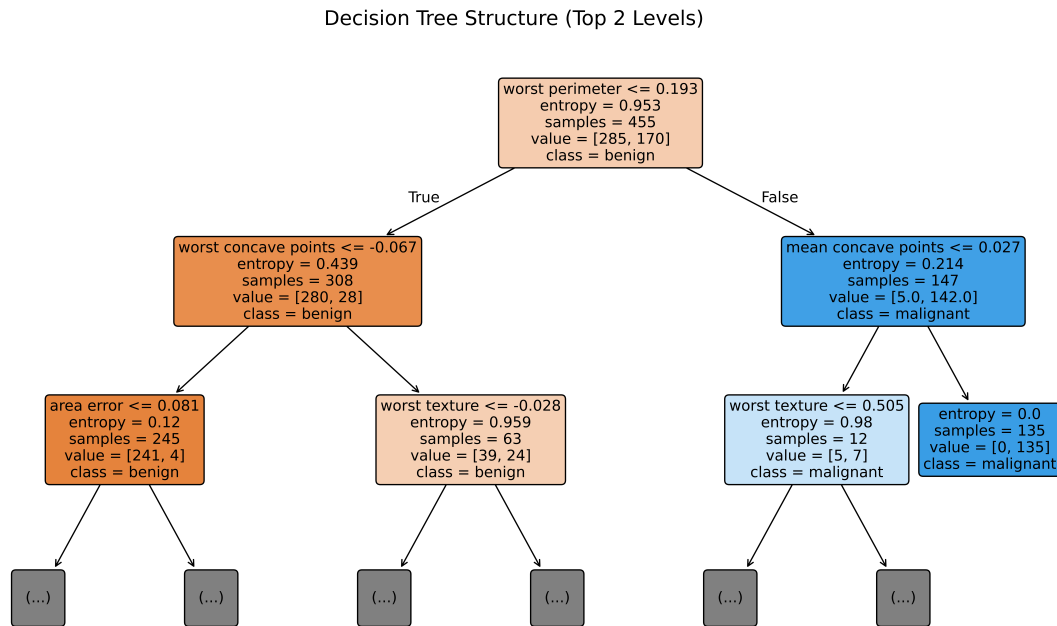


Figure 10: Structure of the tuned Decision Tree (top 2 levels shown for readability; leaf nodes below this depth are collapsed and marked “(...”), exported at 1200 DPI

Inference:

The root node splits on “worst perimeter,” and the very next level splits on “worst concave points” and “mean concave points” – the same features that the Random Forest ranked as most important, which is reassuring cross-validation of their discriminative power between two very different modelling approaches. The right-hand branch (large worst perimeter) reaches a completely pure malignant leaf (135 out of 135 samples) within just two splits, showing that a small number of well-chosen features already separate most of the data cleanly; the remaining depth of the full tree is largely dedicated to resolving a smaller number of harder, borderline cases such as the mixed-class node with 5 malignant and 7 benign samples.

10.7 Hyperparameter Sensitivity

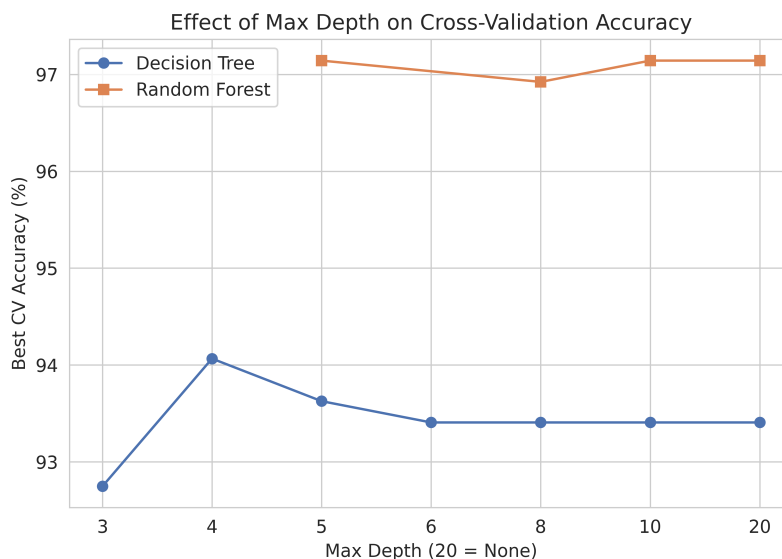


Figure 11: Effect of maximum tree depth on best-achieved cross-validation accuracy for both models

Inference:

The Decision Tree’s accuracy curve peaks early (around depth 4) and then flattens or dips slightly as depth increases further, whereas the Random Forest’s curve stays essentially flat and consistently higher across every depth setting tested. This visually reinforces the central finding of the experiment: depth is a sensitive, overfitting-prone hyperparameter for a single tree, but its effect is largely neutralised once many trees are combined in an ensemble.

11 Discussion

Across every evaluation angle – held-out test metrics, 5-fold cross-validation, ROC/PR analysis, and learning curves – the Random Forest consistently outperformed the single Decision Tree, improving test accuracy from 92.98% to 97.37% and, more importantly for a diagnostic task, improving recall on the malignant class from 80.95% to 92.86%. The Decision Tree’s main weakness was variance: even after tuning `max_depth` and the minimum-samples parameters via cross-validation, it still showed a visible training/validation gap in the learning curves and the widest fold-to-fold swing in cross-validation accuracy among the two models. This matches the theoretical expectation that a single tree, however well-pruned, remains a high-variance estimator.

The Random Forest’s strength came from bagging together many decorrelated trees: the hyperparameter search showed that its cross-validation accuracy was remarkably insensitive to `max_depth`, `n_estimators`, and `max_features`, with nearly every configuration landing within roughly one percentage point of the best one. This stability is itself evidence that ensembling, rather than any single “magic” hyperparameter setting, is what drives the performance gain.

That said, ensemble learning does not *always* guarantee an improvement – it helps most when the base learner (here, a Decision Tree) has high variance and the individual trees in the forest are reasonably decorrelated from one another. On a dataset that is already easy to separate linearly, or where a single well-regularised tree already achieves near-ceiling accuracy, the incremental benefit of an ensemble can be small, while the cost in interpretability and inference time is still paid. In this experiment, the WDBC dataset’s moderate class imbalance and the presence of noisy/outlier feature values are exactly the conditions under which bagging is expected to help the most, which is consistent with the sizeable improvement observed.

A limitation of the current study is that hyperparameter tuning was restricted to a fixed grid; a randomised or Bayesian search over a continuous parameter space might find marginally better configurations, particularly for the Random Forest, where the search already showed a flat performance plateau. Future extensions could also explore boosting-based ensembles (e.g., Gradient Boosting or XGBoost) as an additional point of comparison, and could report calibration metrics (e.g., Brier score) given that the predicted probabilities are used for the ROC and precision-recall analysis.

12 Conclusion

Decision Tree and Random Forest classifiers were implemented and evaluated on the Wisconsin Diagnostic Breast Cancer dataset using 5-fold cross-validation for hyperparameter selection. The tuned Decision Tree (entropy criterion, `max_depth = 4`) achieved 92.98% test accuracy and an F1-score of 89.47%, while the tuned Random Forest (200 trees, `max_depth = 10`, `max_features='sqrt'`) achieved 97.37% test accuracy, an F1-score of 96.30%, and a ROC-AUC of 0.994. The Random Forest reduced variance and improved both stability across cross-validation folds and generalisation to the held-out test set, empirically confirming the theoretical advantage of ensemble bagging over a single decision tree for this classification task.

13 References

- [1] Scikit-learn Developers, “Decision Trees,” *scikit-learn documentation*. [Online]. Available: <https://scikit-learn.org/stable/modules/tree.html>
- [2] Scikit-learn Developers, “Ensemble methods: Random Forests,” *scikit-learn documentation*. [Online]. Available: <https://scikit-learn.org/stable/modules/ensemble.html#random-forests>
- [3] W. N. Street, W. H. Wolberg, and O. L. Mangasarian, “Nuclear feature extraction for breast tumor diagnosis,” in *IS&T/SPIE International Symposium on Electronic Imaging*, 1993.
- [4] UCI Machine Learning Repository, “Breast Cancer Wisconsin (Diagnostic) Data Set.” [Online]. Available: <https://archive.ics.uci.edu>
- [5] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.